

Task 8: OOPS – Polymorphism

Instead of creating separate, rigid code streams for each student type, polymorphism lets you treat both regular Student objects and GraduateStudent objects as if they belong to the same parent type Student.

- Adjust our main method to instantiate a mix of profiles and store them inside a single **Parent Array**. We will then use a single loop to scan the directory, and Java will automatically figure out which honor roll logic rules to apply to each student at runtime.
- We have wrapped the entire application inside an infinite loop (**while (true)**), which allows it to run dynamically for as many students as you want. At the start of each turn, the program asks if you want to add a **Regular Student**, a **Graduate Student**, or **Exit** the system. If you choose to exit, the program safely terminates the loop using the break keyword.

What Changes in This Step:

1. **Dynamic Program Loop (while (true))**: The application runs continuously until explicitly stopped.
2. **User-Driven Object Decisions**: An interactive selection menu lets the user choose the specific object type to manufacture on the fly.
3. **Parent Polymorphic Array**: We build a directory array using the parent blueprint type:

```
Student[] schoolRoll = new Student[2];
```
4. **Dynamic Slots**: Slot 0 is assigned a regular Student object, and slot 1 is assigned a GraduateStudent child object. Java allows this because a graduate student *is* a student. [1]
5. **Runtime Magic**: Inside our processing loop, the exact same method call (`current.qualifiesForHonorRoll()`) switches its underlying engine automatically—applying the 90.0% rule to slot 0 and the stricter 95.0% rule to slot 1 break: Instantly destroys the infinite loop the millisecond choice 3 is entered.
6. **continue**: Safely throws the application back to the menu start if the user inputs an unexpected integer.
7. **instanceof**: A new helper keyword used at the end to check: "Is the student currently passing through the engine a graduate student?" If yes, it safely unlocks and displays their hidden thesis title.

This seems like production-grade console application! Congratulation on First project.

Note:

No changes to class Student and class GraduateStudent for adding polymorphism.

Only adjust main method to instantiate a mix of profiles and store them inside a single **Parent Array**.

```
// — CONTINUOUS MAIN RUNNER EXECUTION SCRIPT —
public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=====");
        System.out.println("  WELCOME TO THE GLOBAL STUDENT ENGINE  ");
        System.out.println("=====");

        // — DYNAMIC CONTINUOUS WHILE LOOP —
        while (true) {
            System.out.println("\nSelect an action:");
            System.out.println("1. Register a Regular Student");
            System.out.println("2. Register a Graduate Student");
            System.out.println("3. Exit Program");
            System.out.print("Enter choice (1-3): ");
            int menuChoice = inputScanner.nextInt();
            inputScanner.nextLine(); // Clear out leftover newline token from scanner buffer

            // — DYNAMIC EXIT JUMP CONDITION —
            if (menuChoice == 3) {
                System.out.println("\nExiting System... Thank you for using Student Engine. Goodbye!");
                break; // Instantly breaks the infinite while loop to end the program safely
            }

            // Route user choices or intercept bad choices
            if (menuChoice != 1 && menuChoice != 2) {
                System.out.println("✗ Invalid option. Please select 1, 2, or 3.");
                continue; // Skips the rest of this iteration and restarts the menu loop
            }

            // GATHER SHARED BASE PROFILE DETAILS
            System.out.print("\nEnter raw student name: ");
            String rawName = inputScanner.nextLine();

            // Declare a generic parent type pointer variable to handle polymorphism seamlessly
            Student currentStudent = null;

            // — USER DECIDES THE INSTANTIATION TYPE —
            if (menuChoice == 1) {
                // Instantiates a regular student profile
                currentStudent = new Student(rawName, inputScanner);
            } else if (menuChoice == 2) {
                System.out.print("Enter Research Thesis Topic: ");
                String thesis = inputScanner.nextLine();
                // Instantiates a specialized graduate student child profile
                currentStudent = new GraduateStudent(rawName, thesis, inputScanner);
            }
        }
    }
}
```

```
// — POLYMORPHIC REPORT COMPILATION —
System.out.println("\n=====");
System.out.println("          GENERATING REPORT CARD          ");
System.out.println("=====");
System.out.println("Account File:   " + currentStudent.getName());
System.out.println("Tracked Classes: " + currentStudent.getGrades().length);

// If the object is a GraduateStudent, show their thesis topic using a safe type cast check
if (currentStudent instanceof GraduateStudent) {
    GraduateStudent castedGrad = (GraduateStudent) currentStudent;
    System.out.println("Research Focus:  " + castedGrad.getThesisTitle());
}

double averageGrade = currentStudent.calculateAverage();
System.out.println("Final Grade:    " + averageGrade + "%");

// Runtime Magic: Automatically triggers the correct 90% or 95% target rules
if (currentStudent.qualifiesForHonorRoll()) {
    System.out.println("Honor Roll:      ★ APPROVED ★");
} else {
    System.out.println("Honor Roll:      Standard Academic Standing");
}
System.out.println("=====");
}

inputScanner.close();
}
}
```